TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void printArray(int arr[], int n) {
    int i;

    printf("[");
    for (i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1) {
            printf(", ");
        }
    }
    printf("]\n");
}

int main() {
    int arr1[] = {0};
```

Console | I/O | AI Agent | 185 ms

```
Test Case 1:
[]
Test Case 2:
[1]
Test Case 3:
[7, 7, 7, 7]
Test Case 4:
[-5, -4, -3, -2, -1]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:08 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c
void printArray(int arr[], int n) {
    for (i = 0; i < n; i++) {
        if (i < n - 1) {
        }
    }
    printf("]\n");
}

int main() {
    int arr1[] = {0};
    int arr2[] = {1};
    int arr3[] = {7, 7, 7, 7};
    int arr4[] = {-5, -1, -3, -2, -4};

    printf("Test Case 1:\n");
    bubbleSort(arr1, 0);
    printArray(arr1, 0);

    printf("Test Case 2:\n");
    bubbleSort(arr2, 1);
    printArray(arr2, 1);

    printf("Test Case 3:\n");
    bubbleSort(arr3, 4);
    printArray(arr3, 4);

    printf("Test Case 4:\n");
    bubbleSort(arr4, 5);
    printArray(arr4, 5);

    return 0;
}
```

Console | I/O | AI Agent | 185 ms

```
Test Case 1:
[]
Test Case 2:
[1]
Test Case 3:
[7, 7, 7, 7]
Test Case 4:
[-5, -4, -3, -2, -1]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:08 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>

void selectionSort(int arr[], int n)
{
    int i, j, minIndex, temp;

    for (i = 0; i < n - 1; i++)
    {
        minIndex = i;

        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }

        temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

void printArray(int arr[], int n)
{
    int i;

    printf("[");
    for (i = 0; i < n; i++)
    {
```

Console | I/O | AI Agent | 186 ms

```
Test Case 1:
[1, 2, 5, 5, 6, 9]
Test Case 2:
[2, 4, 6, 8, 10]
Test Case 3:
[1, 2, 3, 4, 5]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:09 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{

    printf("[");
    for (i = 0; i < n; i++)
    {
        printf("%d", arr[i]);
        if (i < n - 1)
        {
            printf(", ");
        }
    }
    printf("]\n");
}

int main(void)
{
    int arr1[] = {5, 2, 9, 1, 5, 6};
    int arr2[] = {10, 8, 6, 4, 2};
    int arr3[] = {1, 2, 3, 4, 5};

    int n1 = sizeof(arr1) / sizeof(arr1[0]);
    int n2 = sizeof(arr2) / sizeof(arr2[0]);
    int n3 = sizeof(arr3) / sizeof(arr3[0]);

    printf("Test Case 1:\n");
    selectionSort(arr1, n1);
    printArray(arr1, n1);

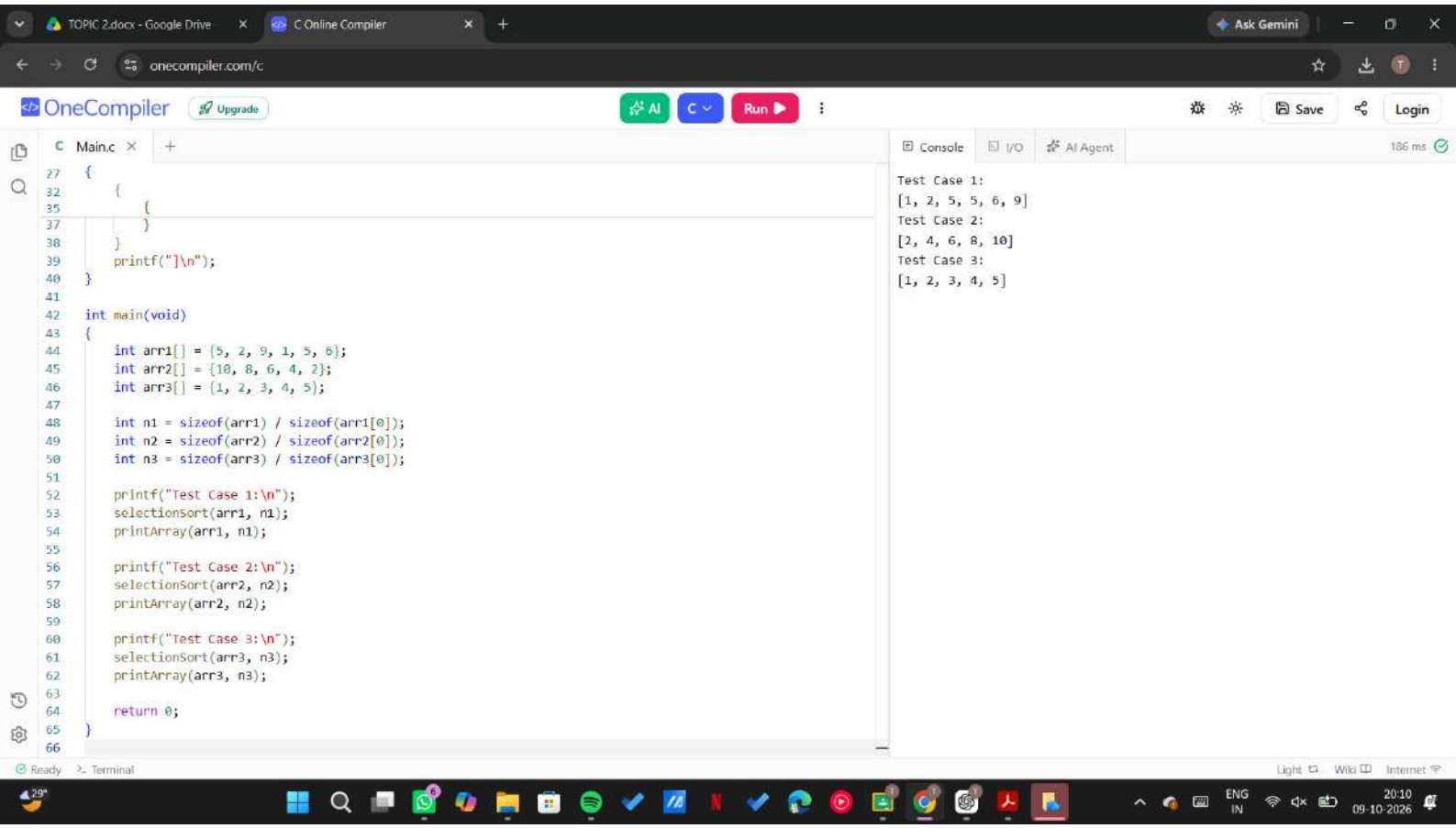
    printf("Test Case 2:\n");
    selectionSort(arr2, n2);
    printArray(arr2, n2);
```

Console | I/O | AI Agent | 186 ms

```
Test Case 1:
[1, 2, 5, 5, 6, 9]
Test Case 2:
[2, 4, 6, 8, 10]
Test Case 3:
[1, 2, 3, 4, 5]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:09 09-10-2026


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
186 ms
{
{
{
}
}
printf("]\n");
}
int main(void)
{
int arr1[] = {5, 2, 9, 1, 5, 6};
int arr2[] = {10, 8, 6, 4, 2};
int arr3[] = {1, 2, 3, 4, 5};
int n1 = sizeof(arr1) / sizeof(arr1[0]);
int n2 = sizeof(arr2) / sizeof(arr2[0]);
int n3 = sizeof(arr3) / sizeof(arr3[0]);
printf("Test Case 1:\n");
selectionSort(arr1, n1);
printArray(arr1, n1);
printf("Test Case 2:\n");
selectionSort(arr2, n2);
printArray(arr2, n2);
printf("Test Case 3:\n");
selectionSort(arr3, n3);
printArray(arr3, n3);
return 0;
}
Test Case 1:
[1, 2, 5, 5, 6, 9]
Test Case 2:
[2, 4, 6, 8, 10]
Test Case 3:
[1, 2, 3, 4, 5]
Ready
Terminal
Light
Wiki
Internet
29°
ENG
IN
20:10
09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    int i, j, temp, swapped;

    for (i = 0; i < n - 1; i++)
    {
        swapped = 0;

        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                swapped = 1;
            }
        }

        if (swapped == 0)
        {
            break;
        }
    }
}

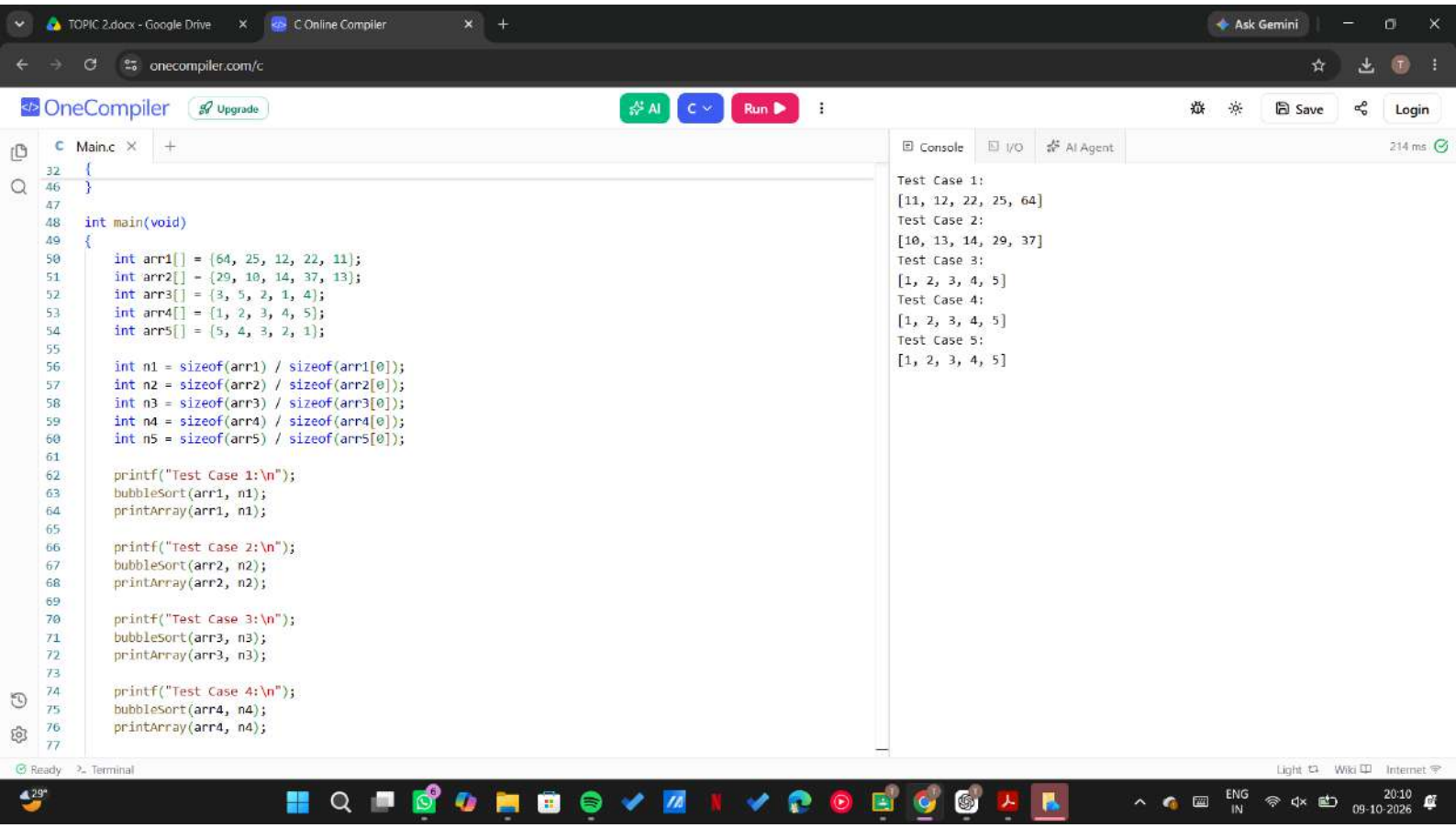
void printArray(int arr[], int n)
{
```

Console | I/O | AI Agent | 214 ms

```
Test Case 1:
[11, 12, 22, 25, 64]
Test Case 2:
[10, 13, 14, 29, 37]
Test Case 3:
[1, 2, 3, 4, 5]
Test Case 4:
[1, 2, 3, 4, 5]
Test Case 5:
[1, 2, 3, 4, 5]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:10 09-10-2026


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
214 ms
{
}
int main(void)
{
int arr1[] = {64, 25, 12, 22, 11};
int arr2[] = {29, 10, 14, 37, 13};
int arr3[] = {3, 5, 2, 1, 4};
int arr4[] = {1, 2, 3, 4, 5};
int arr5[] = {5, 4, 3, 2, 1};
int n1 = sizeof(arr1) / sizeof(arr1[0]);
int n2 = sizeof(arr2) / sizeof(arr2[0]);
int n3 = sizeof(arr3) / sizeof(arr3[0]);
int n4 = sizeof(arr4) / sizeof(arr4[0]);
int n5 = sizeof(arr5) / sizeof(arr5[0]);
printf("Test Case 1:\n");
bubbleSort(arr1, n1);
printArray(arr1, n1);
printf("Test Case 2:\n");
bubbleSort(arr2, n2);
printArray(arr2, n2);
printf("Test Case 3:\n");
bubbleSort(arr3, n3);
printArray(arr3, n3);
printf("Test Case 4:\n");
bubbleSort(arr4, n4);
printArray(arr4, n4);
Test Case 1:
[11, 12, 22, 25, 64]
Test Case 2:
[10, 13, 14, 29, 37]
Test Case 3:
[1, 2, 3, 4, 5]
Test Case 4:
[1, 2, 3, 4, 5]
Test Case 5:
[1, 2, 3, 4, 5]
Ready
Terminal
Light
Wiki
Internet
29°
ENG
IN
20:10
09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    int i, j, temp, swapped;

    for (i = 0; i < n - 1; i++)
    {
        swapped = 0;

        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }

        if (swapped == 0)
        {
            break;
        }
    }
}

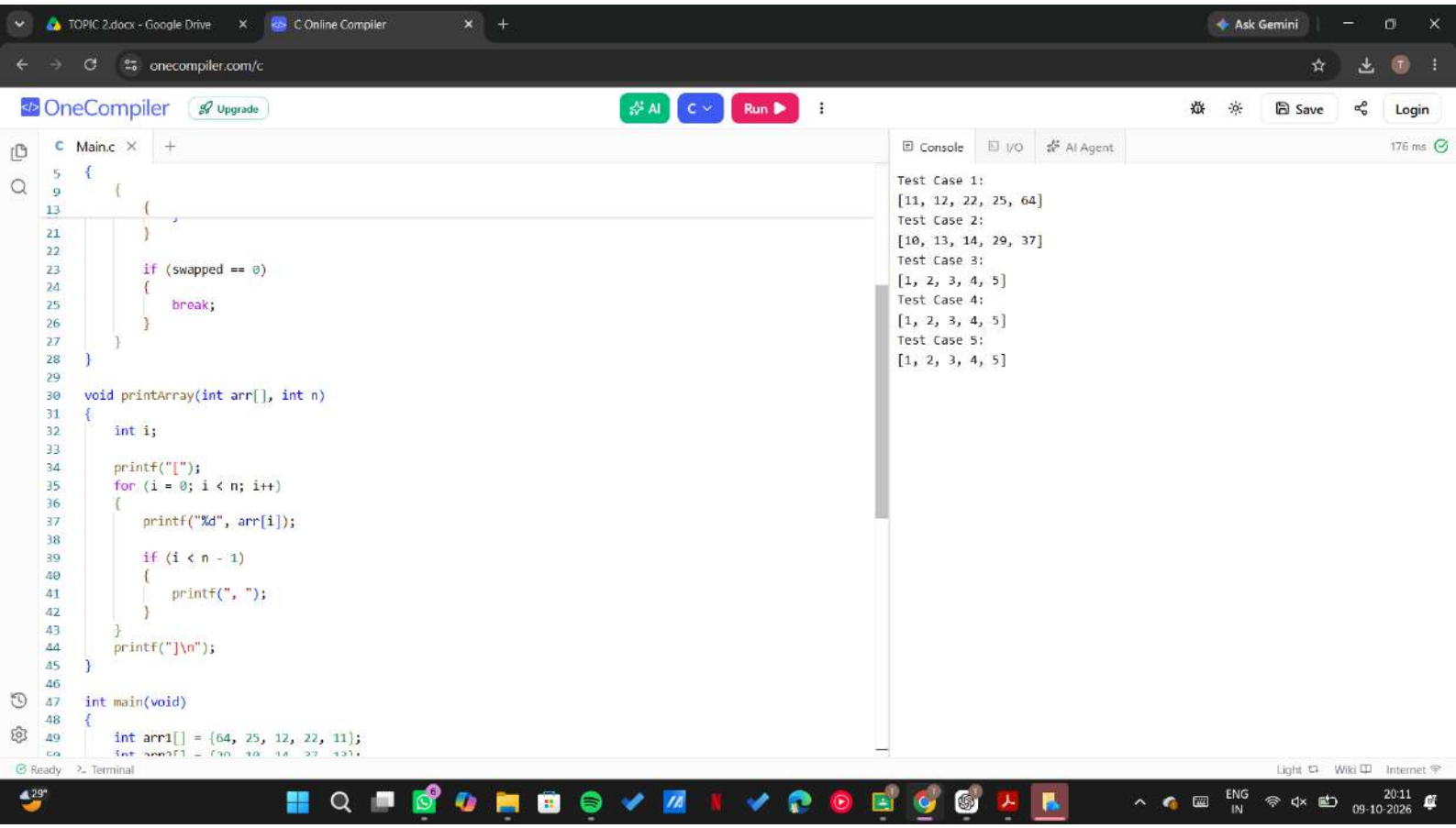
void printArray(int arr[], int n)
{
    int i;
```

Console | I/O | AI Agent | 176 ms

```
Test Case 1:
[11, 12, 22, 25, 64]
Test Case 2:
[10, 13, 14, 29, 37]
Test Case 3:
[1, 2, 3, 4, 5]
Test Case 4:
[1, 2, 3, 4, 5]
Test Case 5:
[1, 2, 3, 4, 5]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:11 09-10-2026


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
176 ms
{
{
{
}
if (swapped == 0)
{
break;
}
}
}
void printArray(int arr[], int n)
{
int i;
printf("[");
for (i = 0; i < n; i++)
{
printf("%d", arr[i]);
if (i < n - 1)
{
printf(", ");
}
}
printf("]\n");
}
int main(void)
{
int arr1[] = {64, 25, 12, 22, 11};
Test Case 1:
[11, 12, 22, 25, 64]
Test Case 2:
[10, 13, 14, 29, 37]
Test Case 3:
[1, 2, 3, 4, 5]
Test Case 4:
[1, 2, 3, 4, 5]
Test Case 5:
[1, 2, 3, 4, 5]
Ready
Terminal
Light
Wiki
Internet
29°
ENG
IN
20:11
09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{
    int arr4[] = {1, 2, 3, 4, 5};
    int arr5[] = {5, 4, 3, 2, 1};

    int n1 = sizeof(arr1) / sizeof(arr1[0]);
    int n2 = sizeof(arr2) / sizeof(arr2[0]);
    int n3 = sizeof(arr3) / sizeof(arr3[0]);
    int n4 = sizeof(arr4) / sizeof(arr4[0]);
    int n5 = sizeof(arr5) / sizeof(arr5[0]);

    printf("Test Case 1:\n");
    bubbleSort(arr1, n1);
    printArray(arr1, n1);

    printf("Test Case 2:\n");
    bubbleSort(arr2, n2);
    printArray(arr2, n2);

    printf("Test Case 3:\n");
    bubbleSort(arr3, n3);
    printArray(arr3, n3);

    printf("Test Case 4:\n");
    bubbleSort(arr4, n4);
    printArray(arr4, n4);

    printf("Test Case 5:\n");
    bubbleSort(arr5, n5);
    printArray(arr5, n5);

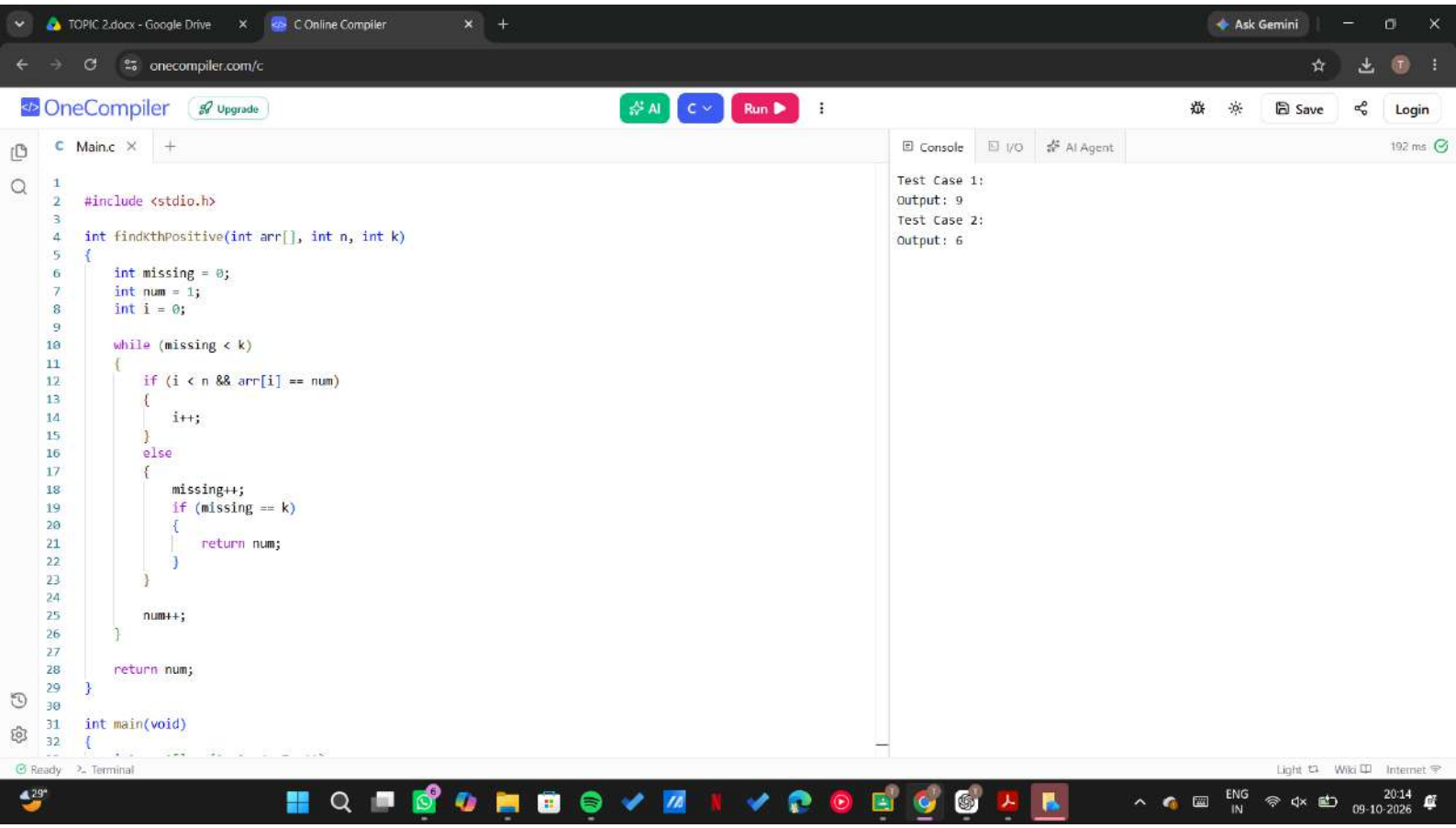
    return 0;
}
```

Console | I/O | AI Agent | 176 ms

```
Test Case 1:
[11, 12, 22, 25, 64]
Test Case 2:
[10, 13, 14, 29, 37]
Test Case 3:
[1, 2, 3, 4, 5]
Test Case 4:
[1, 2, 3, 4, 5]
Test Case 5:
[1, 2, 3, 4, 5]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:11 09-10-2026




```c

#include <stdio.h>

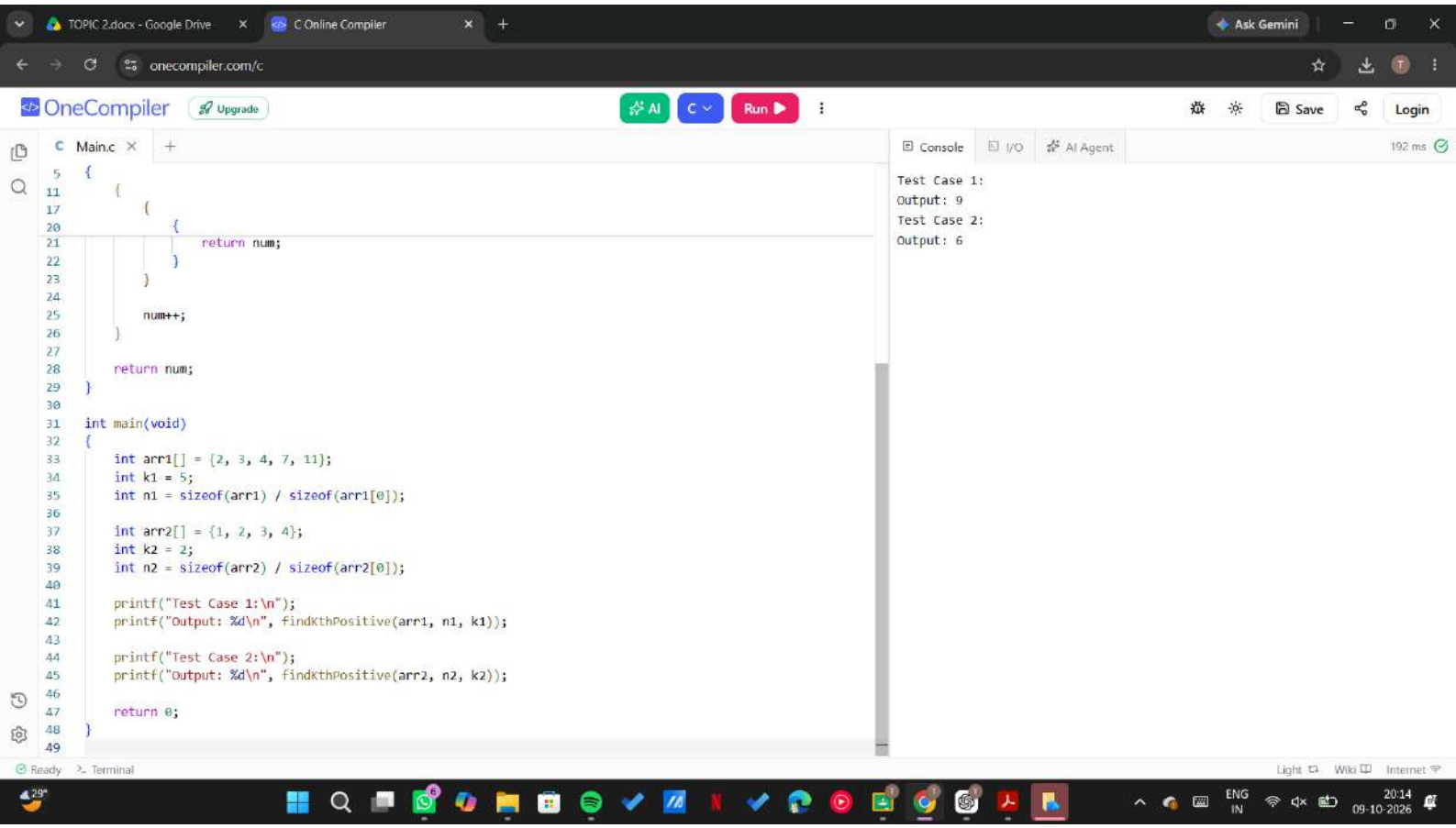
int findkthPositive(int arr[], int n, int k)
{
    int missing = 0;
    int num = 1;
    int i = 0;

    while (missing < k)
    {
        if (i < n && arr[i] == num)
        {
            i++;
        }
        else
        {
            missing++;
            if (missing == k)
            {
                return num;
            }
        }

        num++;
    }

    return num;
}

int main(void)
{
```




```
{
    {
        {
            {
                return num;
            }
        }

        num++;
    }

    return num;
}

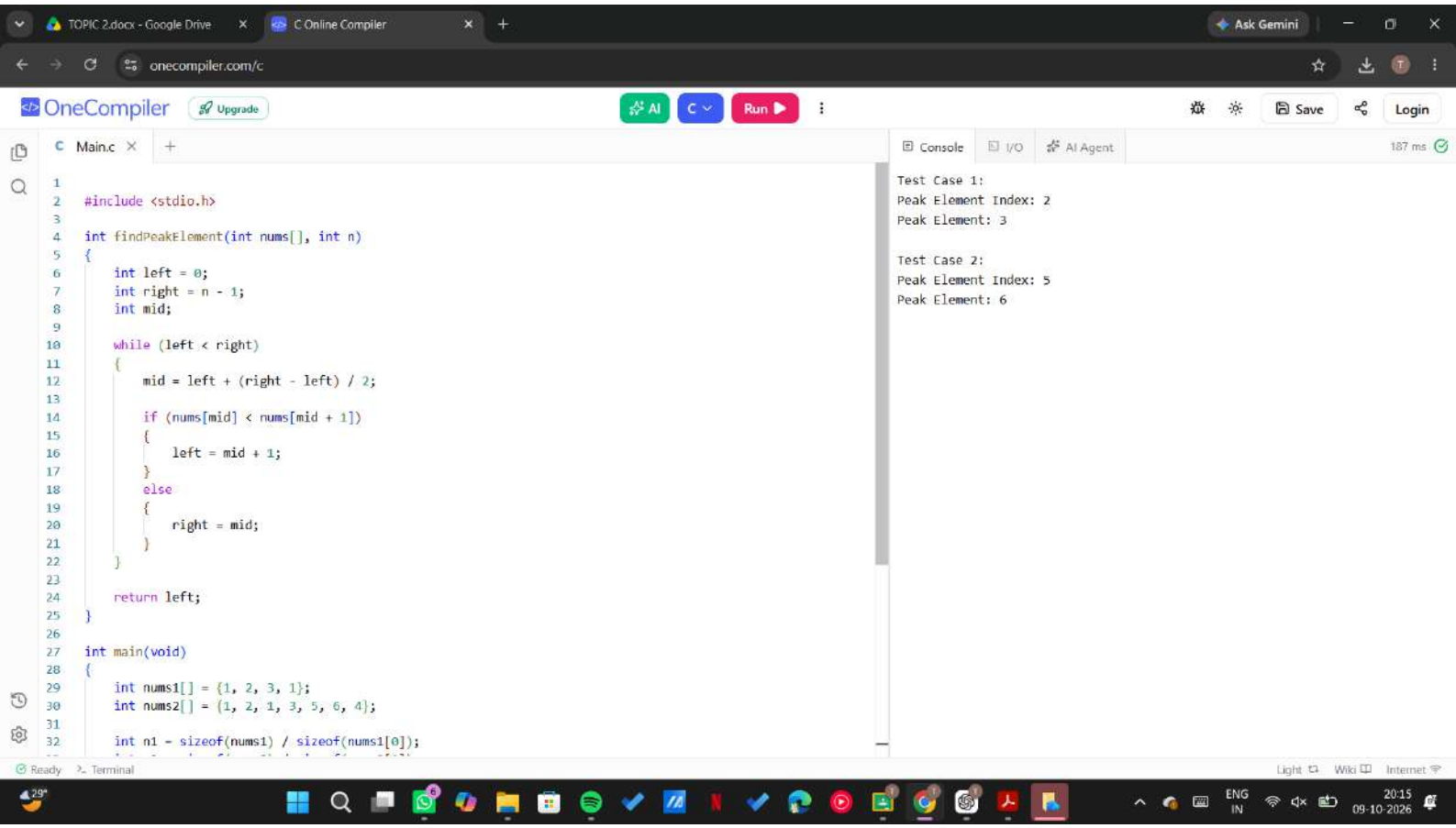
int main(void)
{
    int arr1[] = {2, 3, 4, 7, 11};
    int k1 = 5;
    int n1 = sizeof(arr1) / sizeof(arr1[0]);

    int arr2[] = {1, 2, 3, 4};
    int k2 = 2;
    int n2 = sizeof(arr2) / sizeof(arr2[0]);

    printf("Test Case 1:\n");
    printf("Output: %d\n", findKthPositive(arr1, n1, k1));

    printf("Test Case 2:\n");
    printf("Output: %d\n", findKthPositive(arr2, n2, k2));

    return 0;
}
```




```c
#include <stdio.h>

int findPeakElement(int nums[], int n)
{
    int left = 0;
    int right = n - 1;
    int mid;

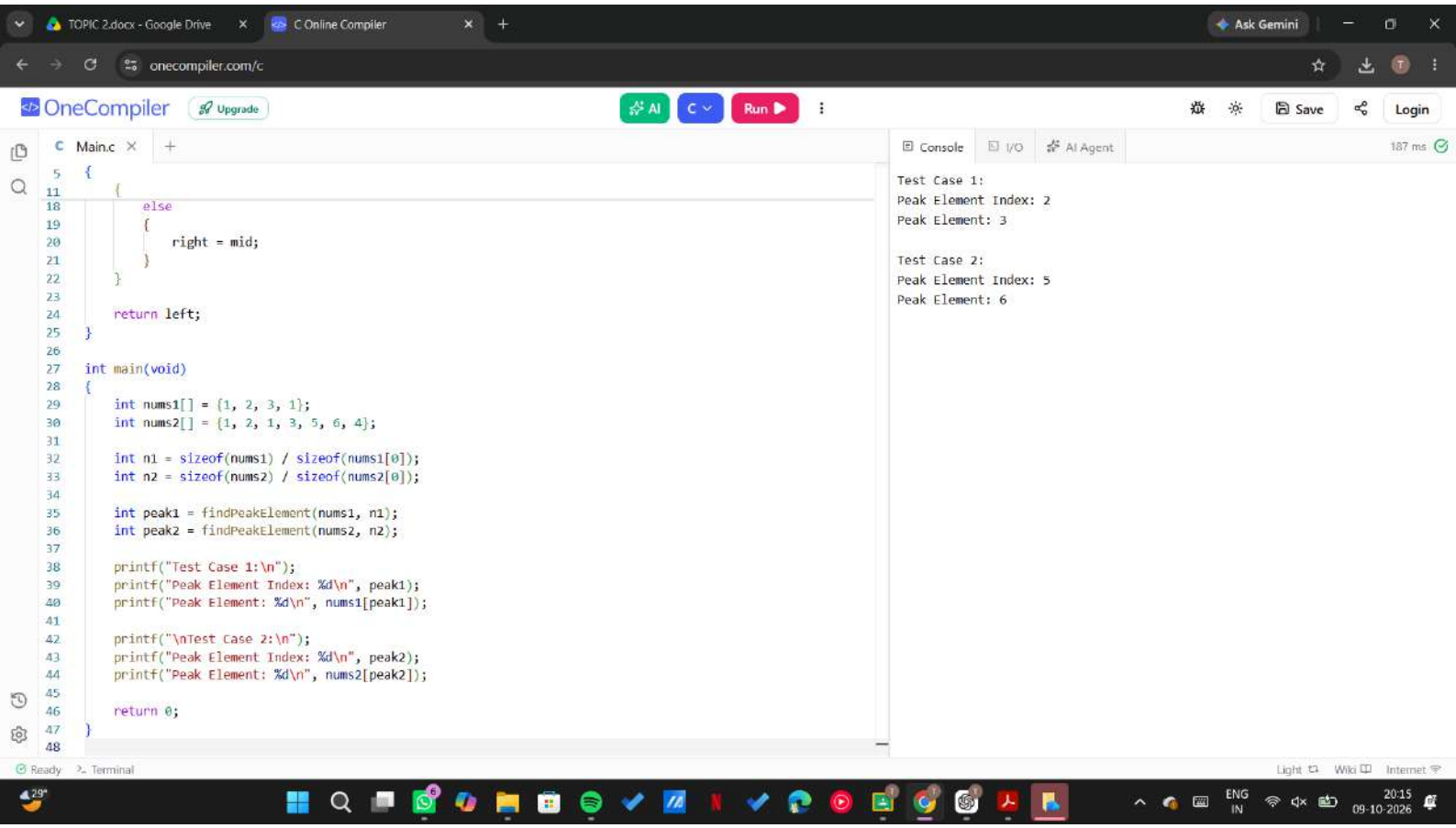
    while (left < right)
    {
        mid = left + (right - left) / 2;

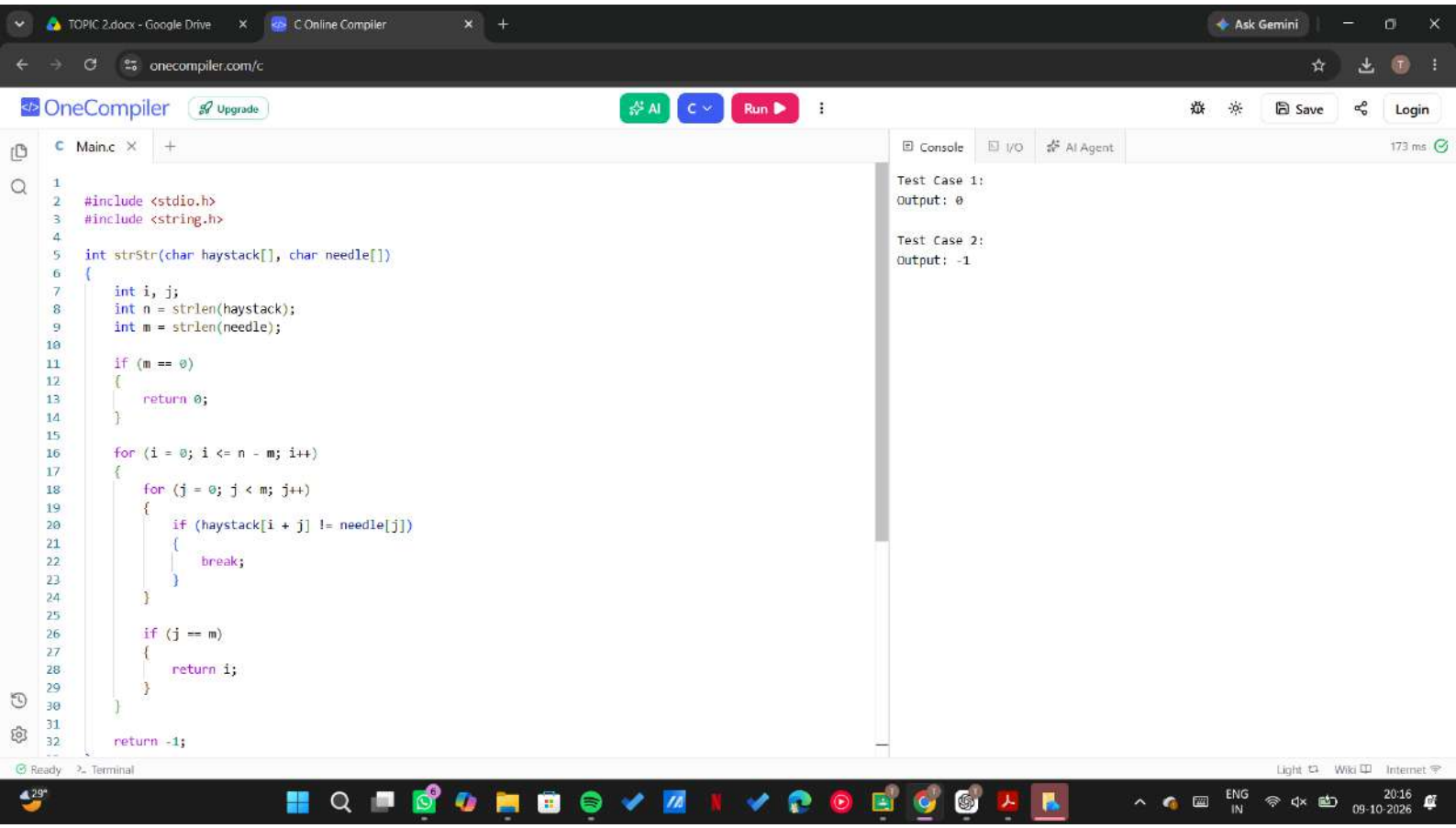
        if (nums[mid] < nums[mid + 1])
        {
            left = mid + 1;
        }
        else
        {
            right = mid;
        }
    }

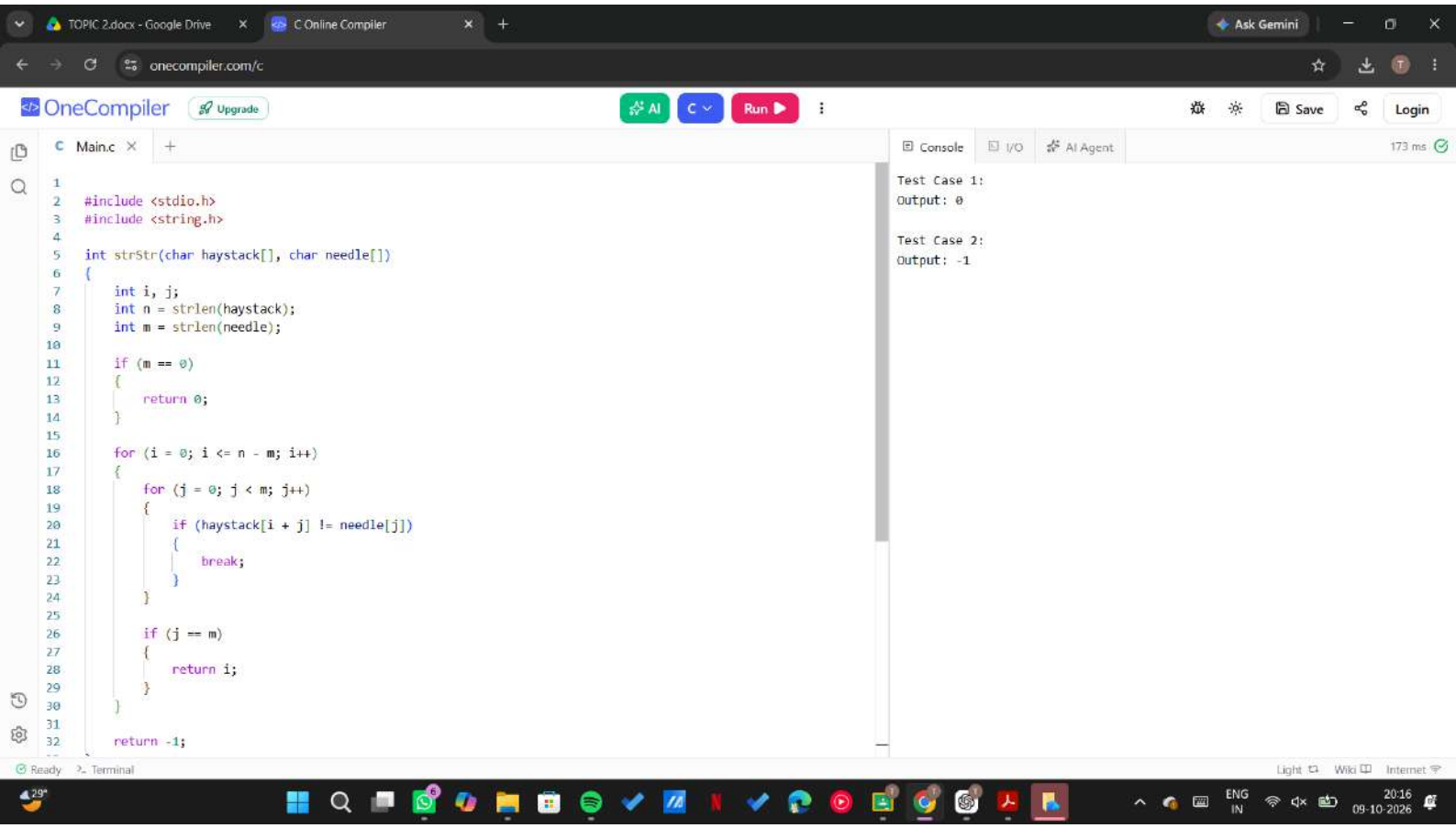
    return left;
}

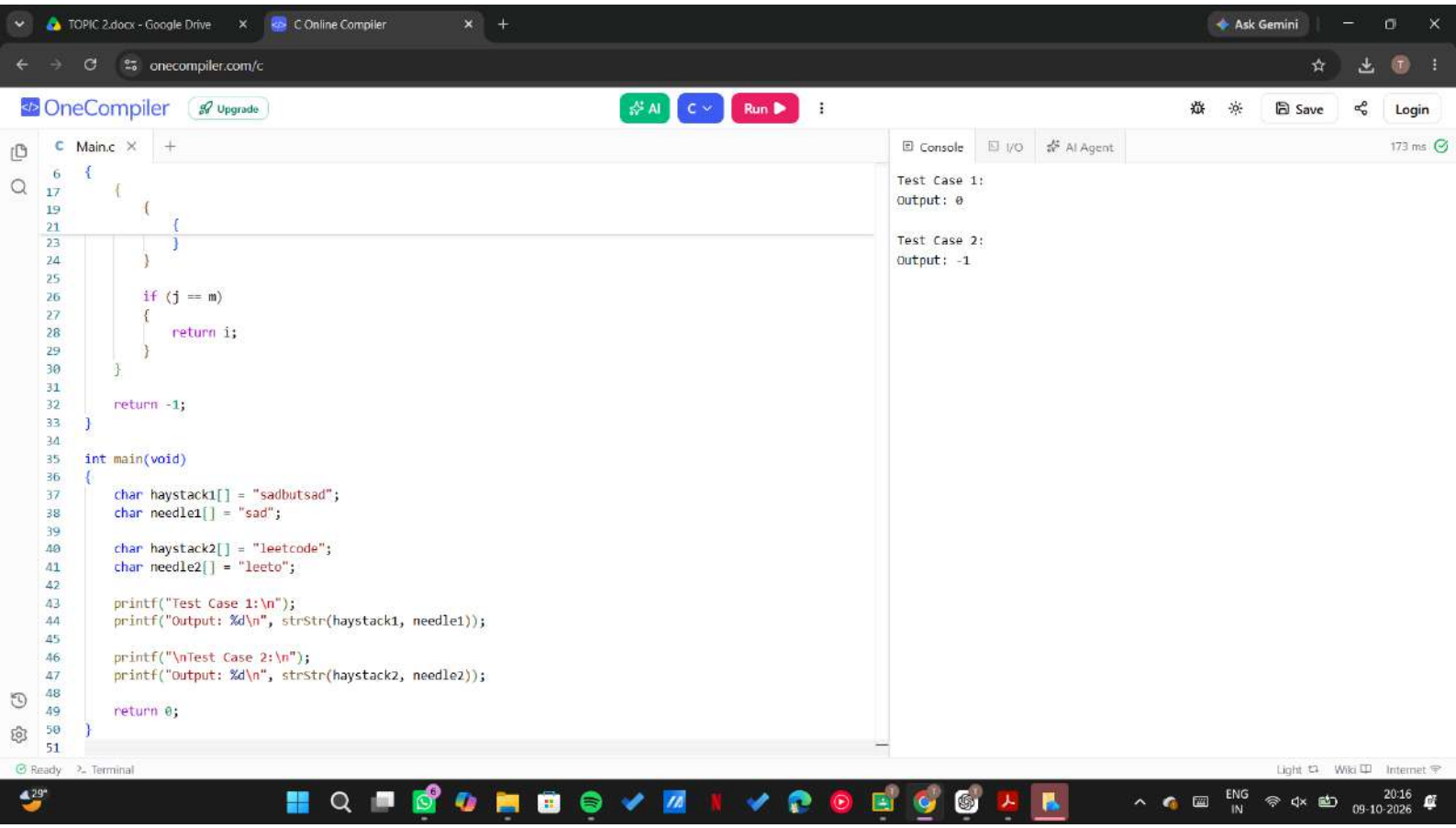
int main(void)
{
    int nums1[] = {1, 2, 3, 1};
    int nums2[] = {1, 2, 1, 3, 5, 6, 4};

    int n1 = sizeof(nums1) / sizeof(nums1[0]);
```


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
187 ms
{
{
else
{
right = mid;
}
}
return left;
}
int main(void)
{
int nums1[] = {1, 2, 3, 1};
int nums2[] = {1, 2, 1, 3, 5, 6, 4};
int n1 = sizeof(nums1) / sizeof(nums1[0]);
int n2 = sizeof(nums2) / sizeof(nums2[0]);
int peak1 = findPeakElement(nums1, n1);
int peak2 = findPeakElement(nums2, n2);
printf("Test Case 1:\n");
printf("Peak Element Index: %d\n", peak1);
printf("Peak Element: %d\n", nums1[peak1]);
printf("\nTest Case 2:\n");
printf("Peak Element Index: %d\n", peak2);
printf("Peak Element: %d\n", nums2[peak2]);
return 0;
}
Test Case 1:
Peak Element Index: 2
Peak Element: 3
Test Case 2:
Peak Element Index: 5
Peak Element: 6
Ready
Terminal
Light
Wiki
Internet
29°
ENG
IN
20:15
09-10-2026


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
173 ms
#include <stdio.h>
#include <string.h>
int strStr(char haystack[], char needle[])
{
int i, j;
int n = strlen(haystack);
int m = strlen(needle);
if (m == 0)
{
return 0;
}
for (i = 0; i <= n - m; i++)
{
for (j = 0; j < m; j++)
{
if (haystack[i + j] != needle[j])
{
break;
}
}
if (j == m)
{
return i;
}
}
return -1;
Test Case 1:
Output: 0
Test Case 2:
Output: -1
Ready
Terminal
Light
Wiki
Internet
ENG
IN
20:16
09-10-2026


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
173 ms
#include <stdio.h>
#include <string.h>
int strStr(char haystack[], char needle[])
{
int i, j;
int n = strlen(haystack);
int m = strlen(needle);
if (m == 0)
{
return 0;
}
for (i = 0; i <= n - m; i++)
{
for (j = 0; j < m; j++)
{
if (haystack[i + j] != needle[j])
{
break;
}
}
if (j == m)
{
return i;
}
}
return -1;
Test Case 1:
Output: 0
Test Case 2:
Output: -1
Ready
Terminal
Light
Wiki
Internet
ENG
IN
20:16
09-10-2026




```
{
    {
        {
            {
            }
        }

        if (j == m)
        {
            return i;
        }
    }

    return -1;
}

int main(void)
{
    char haystack1[] = "sadbutsad";
    char needle1[] = "sad";

    char haystack2[] = "leetcode";
    char needle2[] = "leeto";

    printf("Test Case 1:\n");
    printf("Output: %d\n", strStr(haystack1, needle1));

    printf("\nTest Case 2:\n");
    printf("Output: %d\n", strStr(haystack2, needle2));

    return 0;
}
```

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>
#include <string.h>

int main(void)
{
    char words1[4][20] = {"mass", "as", "hero", "superhero"};
    int n1 = 4;
    int i, j, found;

    printf("Test Case 1:\n");

    for (i = 0; i < n1; i++)
    {
        found = 0;

        for (j = 0; j < n1; j++)
        {
            if (i != j && strstr(words1[j], words1[i]) != NULL)
            {
                found = 1;
                break;
            }
        }

        if (found)
        {
            printf("%s ", words1[i]);
        }
    }

    printf("\n\n");
```

Console | I/O | AI Agent | 229 ms

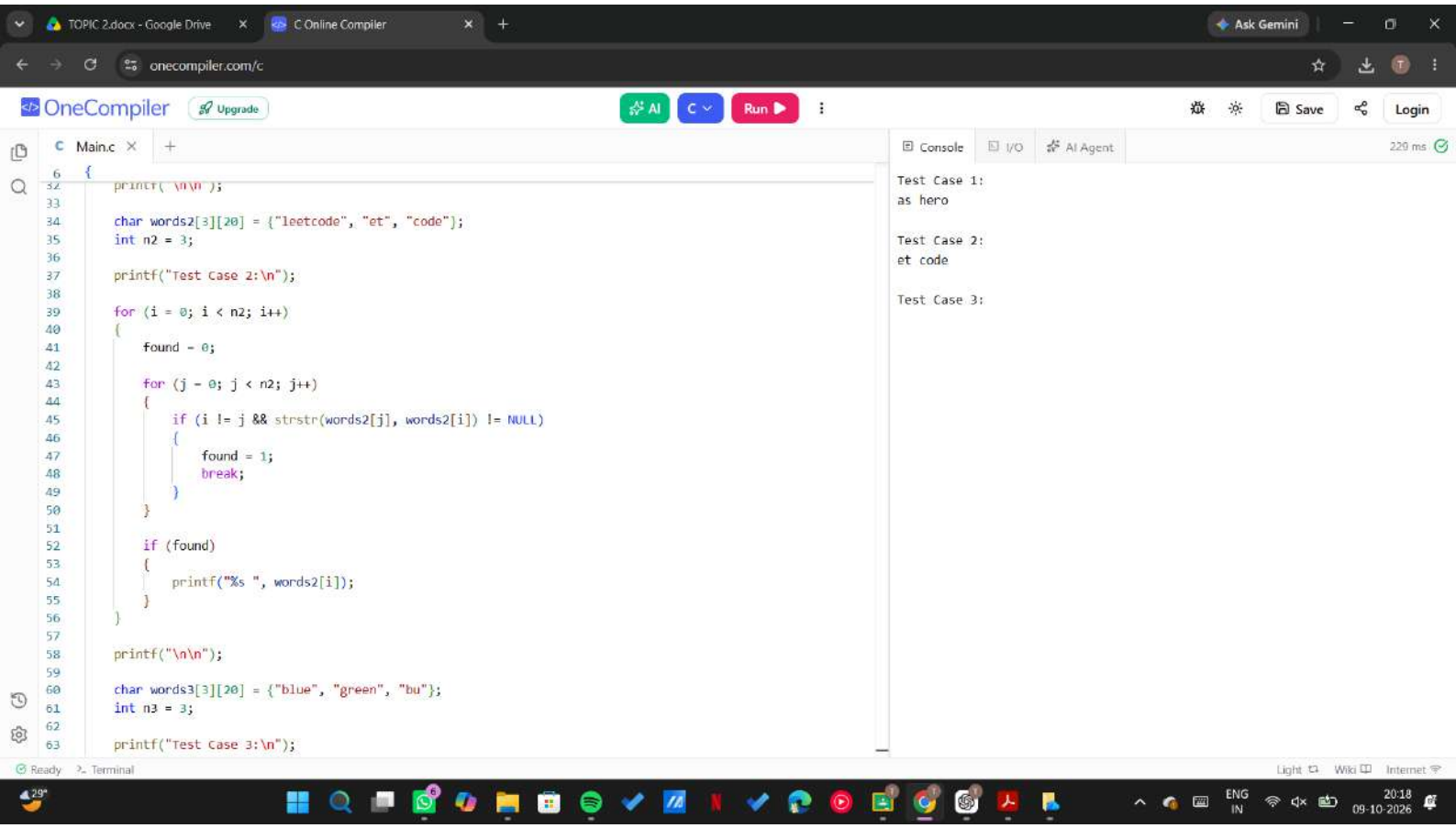
```
Test Case 1:
as hero

Test Case 2:
et code

Test Case 3:
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:18 09-10-2026




```c
{
    printf("\n\n");

    char words2[3][20] = {"leetcode", "et", "code"};
    int n2 = 3;

    printf("Test Case 2:\n");

    for (i = 0; i < n2; i++)
    {
        found = 0;

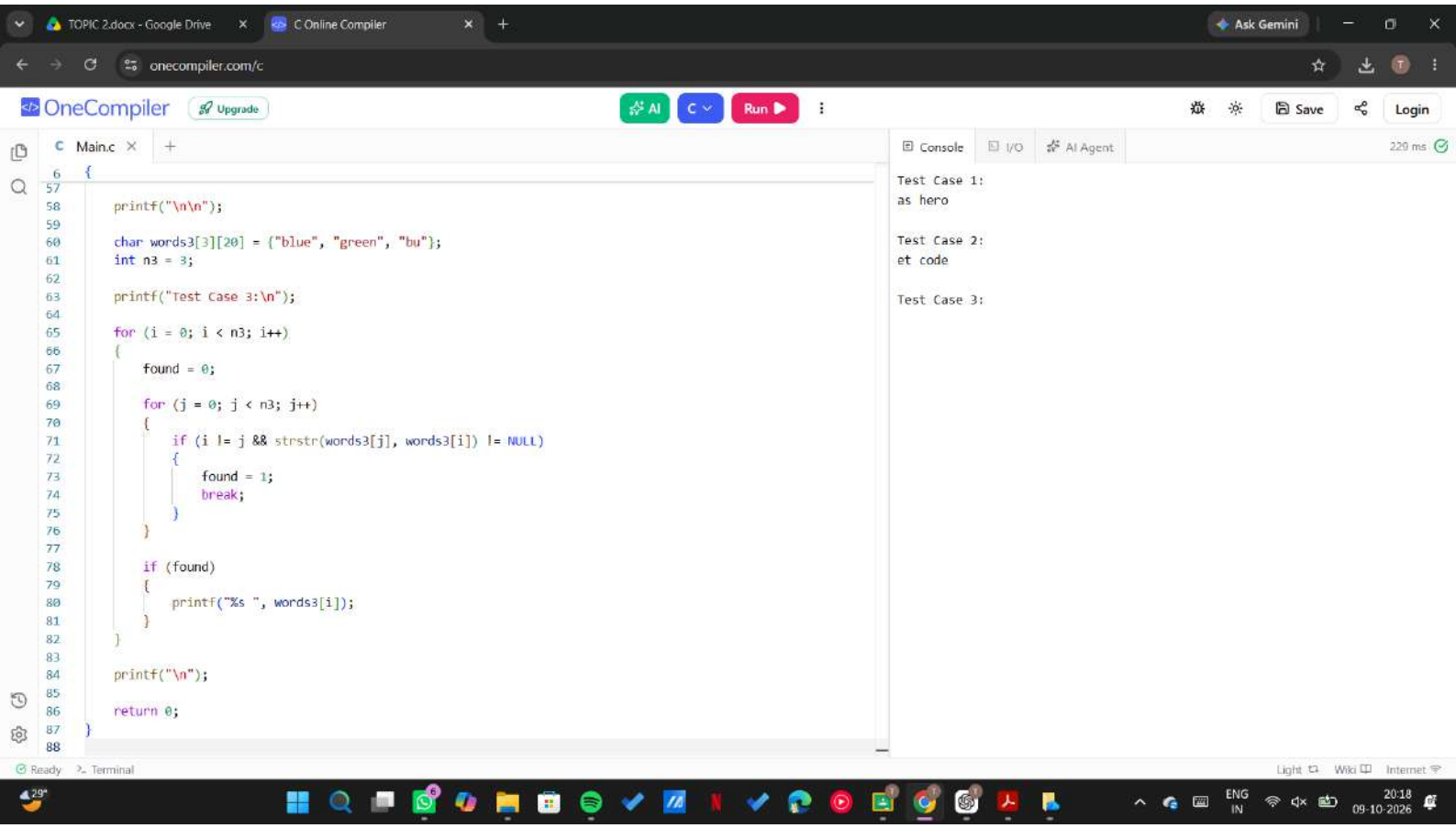
        for (j = 0; j < n2; j++)
        {
            if (i != j && strstr(words2[j], words2[i]) != NULL)
            {
                found = 1;
                break;
            }
        }

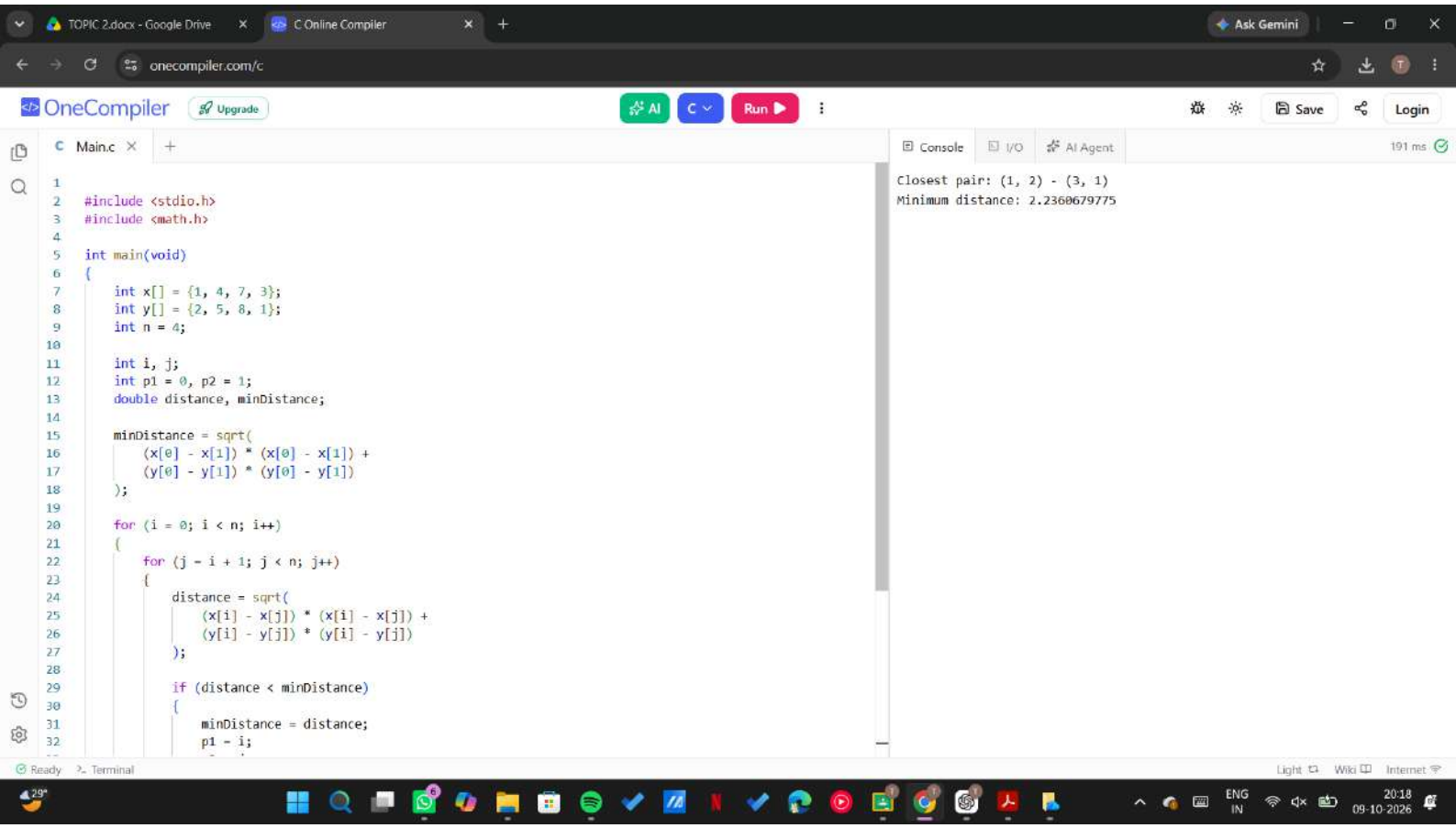
        if (found)
        {
            printf("%s ", words2[i]);
        }
    }

    printf("\n\n");

    char words3[3][20] = {"blue", "green", "bu"};
    int n3 = 3;

    printf("Test Case 3:\n");
```


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
229 ms
{
printf("\n\n");
char words3[3][20] = {"blue", "green", "bu"};
int n3 = 3;
printf("Test Case 3:\n");
for (i = 0; i < n3; i++)
{
found = 0;
for (j = 0; j < n3; j++)
{
if (i != j && strstr(words3[j], words3[i]) != NULL)
{
found = 1;
break;
}
}
if (found)
{
printf("%s ", words3[i]);
}
}
printf("\n");
return 0;
}
Test Case 1:
as hero
Test Case 2:
et code
Test Case 3:
Ready
Terminal
Light
Wiki
Internet
29°
ENG
IN
20:18
09-10-2026




```c

#include <stdio.h>
#include <math.h>

int main(void)
{
    int x[] = {1, 4, 7, 3};
    int y[] = {2, 5, 8, 1};
    int n = 4;

    int i, j;
    int p1 = 0, p2 = 1;
    double distance, minDistance;

    minDistance = sqrt(
        (x[0] - x[1]) * (x[0] - x[1]) +
        (y[0] - y[1]) * (y[0] - y[1])
    );

    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            distance = sqrt(
                (x[i] - x[j]) * (x[i] - x[j]) +
                (y[i] - y[j]) * (y[i] - y[j])
            );

            if (distance < minDistance)
            {
                minDistance = distance;
                p1 = i;
```

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{

    minDistance = sqrt(
        (x[0] - x[1]) * (x[0] - x[1]) +
        (y[0] - y[1]) * (y[0] - y[1])
    );

    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            distance = sqrt(
                (x[i] - x[j]) * (x[i] - x[j]) +
                (y[i] - y[j]) * (y[i] - y[j])
            );

            if (distance < minDistance)
            {
                minDistance = distance;
                p1 = i;
                p2 = j;
            }
        }
    }

    printf("Closest pair: (%d, %d) - (%d, %d)\n",
           x[p1], y[p1], x[p2], y[p2]);

    printf("Minimum distance: %.10f\n", minDistance);

    return 0;
}
```

Console | I/O | AI Agent | 191 ms

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.2360679775
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:19 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>
#include <math.h>

#define N 4
#define EPS 1e-9

typedef struct {
    double x, y;
} Point;

double distance(Point a, Point b)
{
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

void closestPair(Point p[], int n)
{
    int i, j, a = 0, b = 1;
    double d, minDist = distance(p[0], p[1]);

    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            d = distance(p[i], p[j]);

            if (d < minDist) {
                minDist = d;
                a = i;
                b = j;
            }
```

Console | I/O | AI Agent | 179 ms

```
PART A: CLOSEST PAIR
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.2360679775

PART B: CONVEX HULL
Convex Hull points:
(10.0, 0.0)
(5.0, 3.0)
(12.5, 7.0)
(6.0, 6.5)
(7.5, 4.5)
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:19 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{
    }

    printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n",
           p[a].x, p[a].y, p[b].x, p[b].y);
    printf("Minimum distance: %.10f\n", minDist);
}

double cross(Point a, Point b, Point c)
{
    return (b.x - a.x) * (c.y - a.y)
         - (b.y - a.y) * (c.x - a.x);
}

void convexHull(Point p[], int n)
{
    int i, j, k;
    int hull[N] = {0};
    int count = 0;

    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            int positive = 0, negative = 0;

            for (k = 0; k < n; k++) {
                double c;

                if (k == i || k == j)
                    continue;

                c = cross(p[i], p[j], p[k]);

                if (c > EPS)
```

Console | I/O | AI Agent | 179 ms

```
PART A: CLOSEST PAIR
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.2360679775

PART B: CONVEX HULL
Convex Hull points:
(10.0, 0.0)
(5.0, 3.0)
(12.5, 7.0)
(6.0, 6.5)
(7.5, 4.5)
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:19 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
48  {
53      for (i = 0; i < n; i++) {
54          for (j = i + 1; j < n; j++) {
57              for (k = 0; k < n; k++) {
65                  if (c > EPS)
66                      positive = 1;
67                  else if (c < -EPS)
68                      negative = 1;
69              }
70
71              if (!(positive && negative)) {
72                  hull[i] = 1;
73                  hull[j] = 1;
74              }
75          }
76      }
77
78      printf("Convex Hull points:\n");
79      for (i = 0; i < n; i++) {
80          if (hull[i]) {
81              printf("(%.1f, %.1f)\n", p[i].x, p[i].y);
82              count++;
83          }
84      }
85  }
86
87  int main(void)
88  {
89      Point points1[] = {
90          {1, 2}, {4, 5}, {7, 8}, {3, 1}
91      };
92
93      Point points2[] = {
```

Console | I/O | AI Agent | 179 ms

```
PART A: CLOSEST PAIR
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.2360679775

PART B: CONVEX HULL
Convex Hull points:
(10.0, 0.0)
(5.0, 3.0)
(12.5, 7.0)
(6.0, 6.5)
(7.5, 4.5)
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:20 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
48  {
53      for (i = 0; i < n; i++) {
76      }
77
78      printf("Convex Hull points:\n");
79      for (i = 0; i < n; i++) {
80          if (hull[i]) {
81              printf("(%.1f, %.1f)\n", p[i].x, p[i].y);
82              count++;
83          }
84      }
85  }
86
87  int main(void)
88  {
89      Point points1[] = {
90          {1, 2}, {4, 5}, {7, 8}, {3, 1}
91      };
92
93      Point points2[] = {
94          {10, 0}, {11, 5}, {5, 3}, {9, 3.5},
95          {15, 3}, {12.5, 7}, {6, 6.5}, {7.5, 4.5}
96      };
97
98      printf("PART A: CLOSEST PAIR\n");
99      closestPair(points1, 4);
100
101     printf("\nPART B: CONVEX HULL\n");
102     convexHull(points2, 8);
103
104     return 0;
105 }
106
```

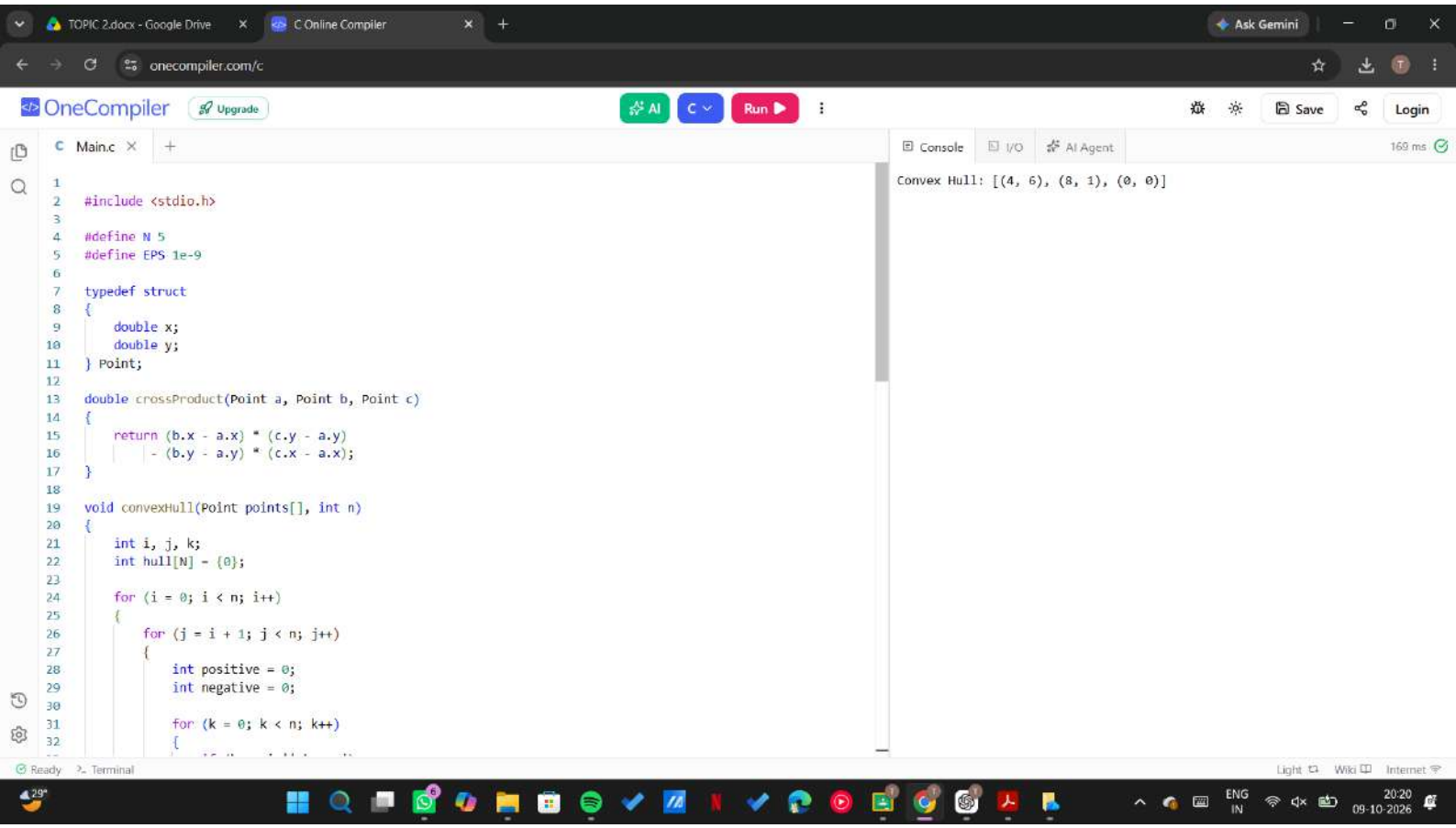
Console | I/O | AI Agent | 179 ms

```
PART A: CLOSEST PAIR
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.2360679775

PART B: CONVEX HULL
Convex Hull points:
(10.0, 0.0)
(5.0, 3.0)
(12.5, 7.0)
(6.0, 6.5)
(7.5, 4.5)
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:20 09-10-2026




```c

#include <stdio.h>

#define N 5
#define EPS 1e-9

typedef struct
{
    double x;
    double y;
} Point;

double crossProduct(Point a, Point b, Point c)
{
    return (b.x - a.x) * (c.y - a.y)
        - (b.y - a.y) * (c.x - a.x);
}

void convexHull(Point points[], int n)
{
    int i, j, k;
    int hull[N] = {0};

    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            int positive = 0;
            int negative = 0;

            for (k = 0; k < n; k++)
            {
```

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{
    {
        {
            {
                if (k == i || k == j)
                double cross = crossProduct(
                    points[i], points[j], points[k]
                );

                if (cross > EPS)
                    positive = 1;
                else if (cross < -EPS)
                    negative = 1;
            }

            if (!(positive && negative))
            {
                hull[i] = 1;
                hull[j] = 1;
            }
        }
    }

    printf("Convex Hull: [");

    int first = 1;

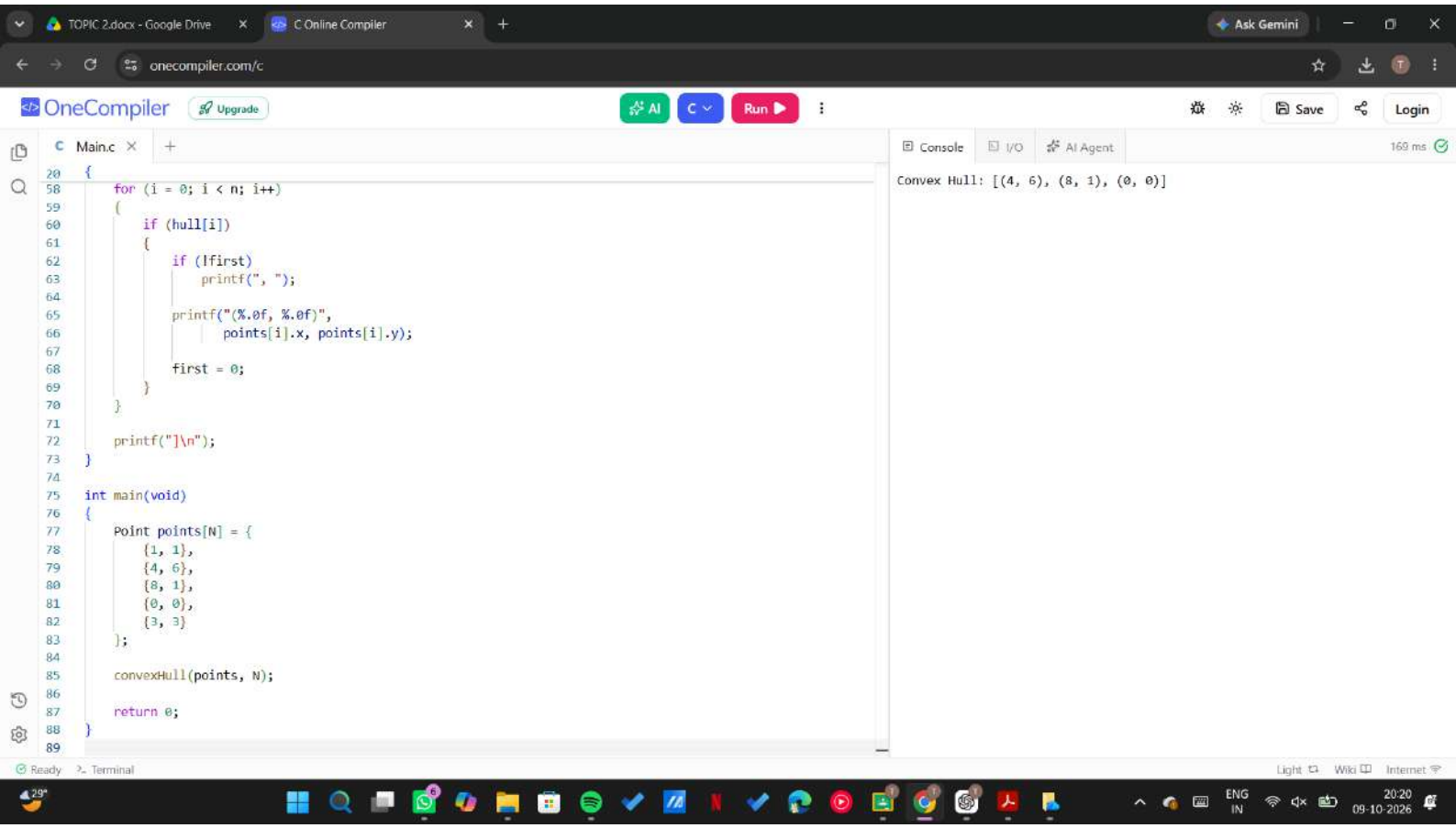
    for (i = 0; i < n; i++)
    {
        if (hull[i])
        {
            if (!first)
```

Console | I/O | AI Agent | 169 ms

```
Convex Hull: [(4, 6), (8, 1), (0, 0)]
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:20 09-10-2026




```c
{
    for (i = 0; i < n; i++)
    {
        if (hull[i])
        {
            if (!first)
                printf(", ");

            printf("(%.0f, %.0f)",
                   points[i].x, points[i].y);

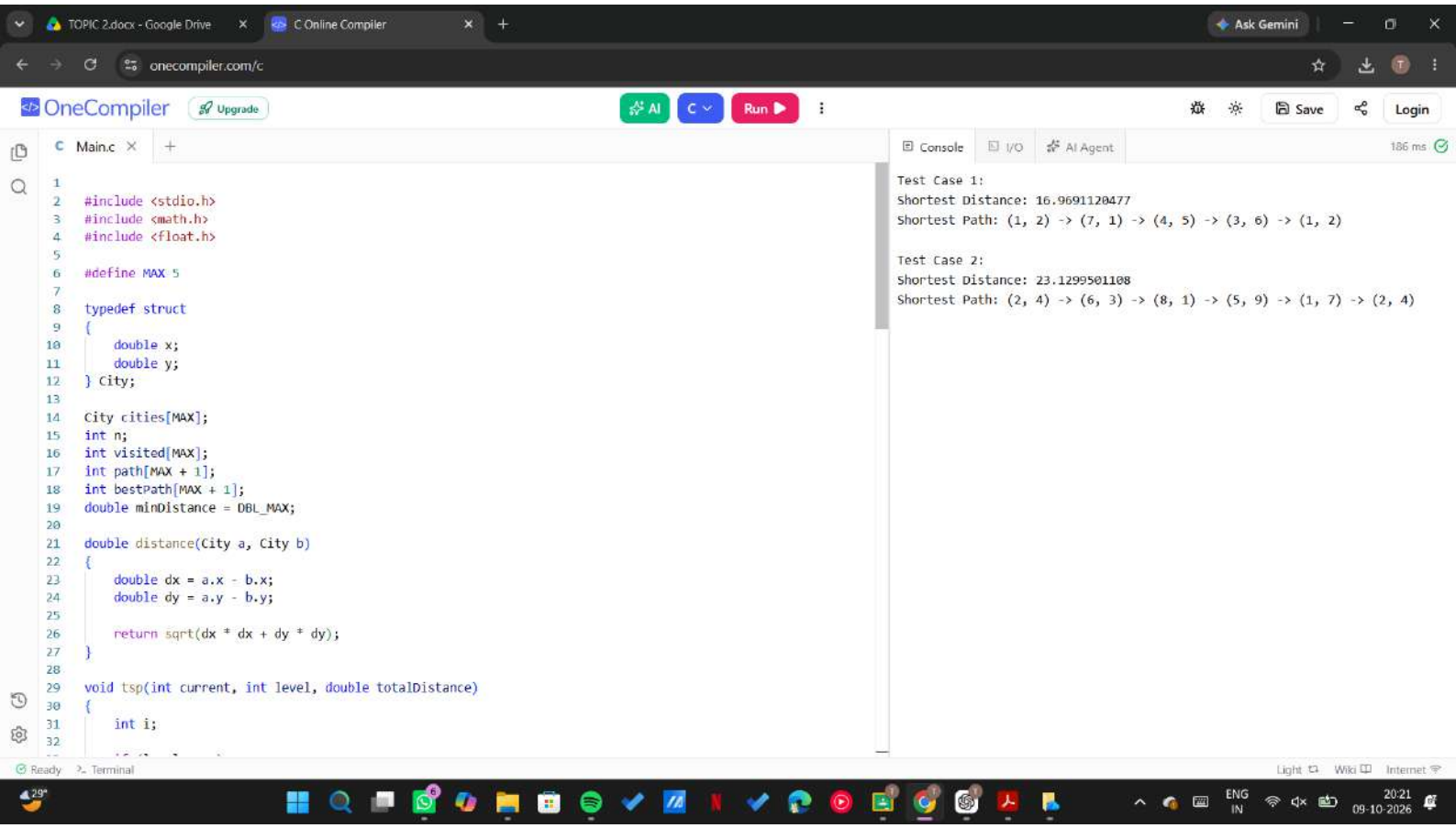
            first = 0;
        }
    }

    printf("]\n");
}

int main(void)
{
    Point points[N] = {
        {1, 1},
        {4, 6},
        {8, 1},
        {0, 0},
        {3, 3}
    };

    convexHull(points, N);

    return 0;
}
```




```c

#include <stdio.h>
#include <math.h>
#include <float.h>

#define MAX 5

typedef struct
{
    double x;
    double y;
} City;

City cities[MAX];
int n;
int visited[MAX];
int path[MAX + 1];
int bestPath[MAX + 1];
double minDistance = DBL_MAX;

double distance(City a, City b)
{
    double dx = a.x - b.x;
    double dy = a.y - b.y;

    return sqrt(dx * dx + dy * dy);
}

void tsp(int current, int level, double totalDistance)
{
    int i;

```

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{
    if (level == n)
    {
        double total = totalDistance + distance(cities[current], cities[0]);

        if (total < minDistance)
        {
            minDistance = total;

            for (i = 0; i < n; i++)
                bestPath[i] = path[i];

            bestPath[n] = 0;
        }

        return;
    }

    for (i = 1; i < n; i++)
    {
        if (!visited[i])
        {
            visited[i] = 1;
            path[level] = i;

            tsp(i, level + 1,
                totalDistance + distance(cities[current], cities[i]));

            visited[i] = 0;
        }
    }
}

```

Console | I/O | AI Agent | 186 ms

```
Test Case 1:
Shortest Distance: 16.9691120477
Shortest Path: (1, 2) -> (7, 1) -> (4, 5) -> (3, 6) -> (1, 2)

Test Case 2:
Shortest Distance: 23.1299501108
Shortest Path: (2, 4) -> (6, 3) -> (8, 1) -> (5, 9) -> (1, 7) -> (2, 4)
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:21 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{
}

void solveTSP(city input[], int size)
{
    int i;

    n = size;
    minDistance = DBL_MAX;

    for (i = 0; i < n; i++)
    {
        cities[i] = input[i];
        visited[i] = 0;
    }

    visited[0] = 1;
    path[0] = 0;

    tsp(0, 1, 0.0);

    printf("Shortest Distance: %.10f\n", minDistance);
    printf("Shortest Path: ");

    for (i = 0; i <= n; i++)
    {
        printf("(%.0f, %.0f)",
               cities[bestPath[i]].x,
               cities[bestPath[i]].y);

        if (i < n)
            printf(" -> ");
    }
```

Console | I/O | AI Agent | 186 ms

```
Test Case 1:
Shortest Distance: 16.9691120477
Shortest Path: (1, 2) -> (7, 1) -> (4, 5) -> (3, 6) -> (1, 2)

Test Case 2:
Shortest Distance: 23.1299501108
Shortest Path: (2, 4) -> (6, 3) -> (8, 1) -> (5, 9) -> (1, 7) -> (2, 4)
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:21 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{
    for (i = 0; i <= n; i++)
    {
        printf("(%.0f, %.0f)",
               cities[bestPath[i]].x,
               cities[bestPath[i]].y);

        if (i < n)
            printf(" -> ");
    }

    printf("\n\n");
}

int main(void)
{
    City test1[] = {
        {1, 2}, {4, 5}, {7, 1}, {3, 6}
    };

    City test2[] = {
        {2, 4}, {8, 1}, {1, 7}, {6, 3}, {5, 9}
    };

    printf("Test Case 1:\n");
    solveTSP(test1, 4);

    printf("Test Case 2:\n");
    solveTSP(test2, 5);

    return 0;
}
```

Console | I/O | AI Agent | 186 ms

```
Test Case 1:
Shortest Distance: 16.9691120477
Shortest Path: (1, 2) -> (7, 1) -> (4, 5) -> (3, 6) -> (1, 2)

Test Case 2:
Shortest Distance: 23.1299501108
Shortest Path: (2, 4) -> (6, 3) -> (8, 1) -> (5, 9) -> (1, 7) -> (2, 4)
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:21 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>
#include <limits.h>

#define N 3

int cost[N][N];
int visited[N];
int currentAssignment[N];
int bestAssignment[N];
int minCost = INT_MAX;

void assignmentProblem(int worker, int totalCost)
{
    int task, i;

    if (worker == N)
    {
        if (totalCost < minCost)
        {
            minCost = totalCost;

            for (i = 0; i < N; i++)
                bestAssignment[i] = currentAssignment[i];
        }

        return;
    }

    for (task = 0; task < N; task++)
    {
        if (!visited[task])
```

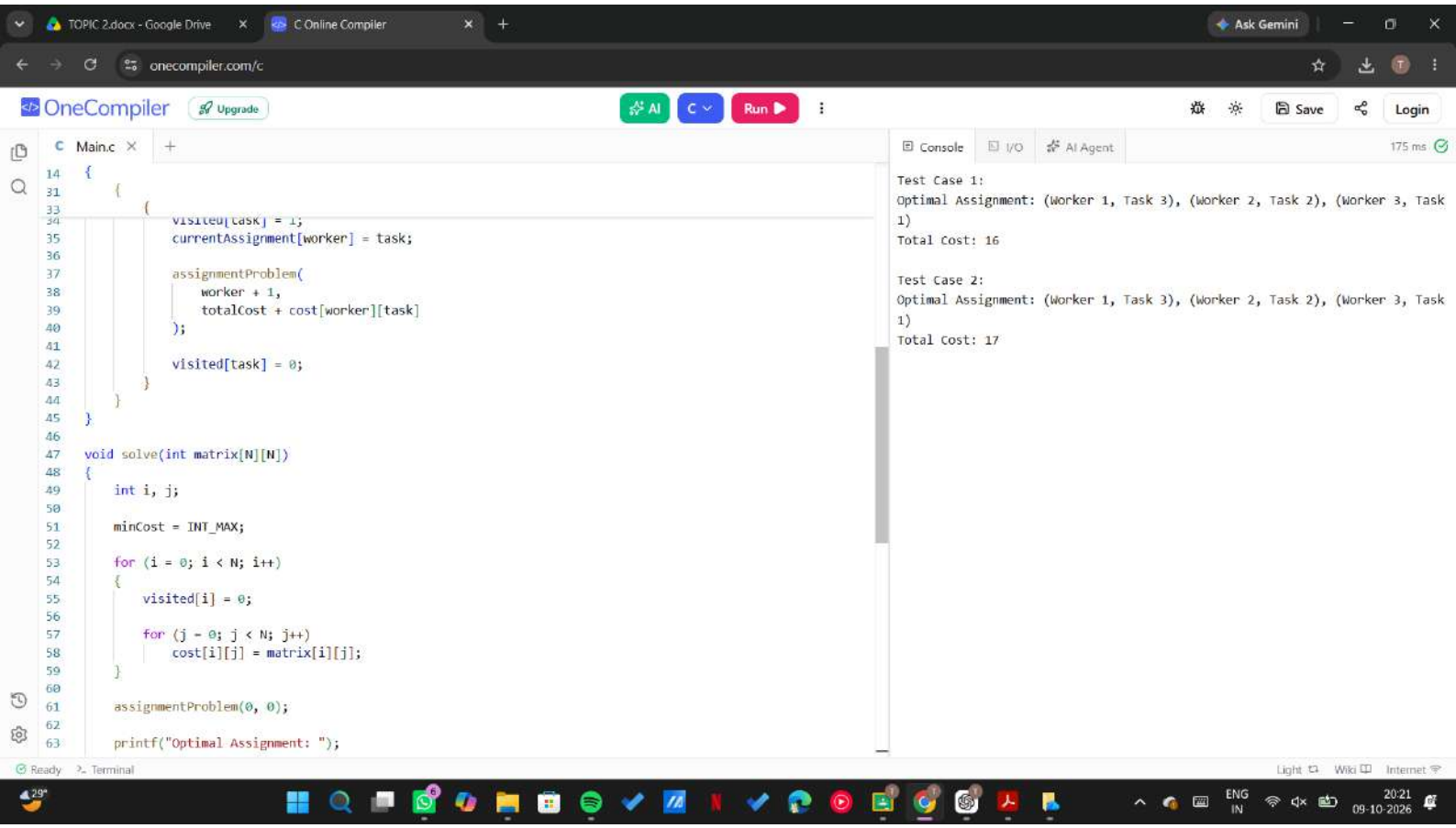
Console | I/O | AI Agent | 175 ms

```
Test Case 1:
Optimal Assignment: (Worker 1, Task 3), (Worker 2, Task 2), (Worker 3, Task 1)
Total Cost: 16

Test Case 2:
Optimal Assignment: (Worker 1, Task 3), (Worker 2, Task 2), (Worker 3, Task 1)
Total Cost: 17
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:21 09-10-2026


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
175 ms
{
{
{
visited[task] = 1;
currentAssignment[worker] = task;
assignmentProblem(
worker + 1,
totalCost + cost[worker][task]
);
visited[task] = 0;
}
}
}
void solve(int matrix[N][N])
{
int i, j;
minCost = INT_MAX;
for (i = 0; i < N; i++)
{
visited[i] = 0;
for (j = 0; j < N; j++)
cost[i][j] = matrix[i][j];
}
assignmentProblem(0, 0);
printf("Optimal Assignment: ");
Test Case 1:
Optimal Assignment: (Worker 1, Task 3), (Worker 2, Task 2), (Worker 3, Task 1)
Total Cost: 16
Test Case 2:
Optimal Assignment: (Worker 1, Task 3), (Worker 2, Task 2), (Worker 3, Task 1)
Total Cost: 17
Ready
Terminal
Light
Wiki
Internet
29°
ENG
IN
20:21
09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
{

    for (i = 0; i < N; i++)
    {
        printf("(Worker %d, Task %d)",
               i + 1, bestAssignment[i] + 1);

        if (i < N - 1)
            printf(", ");
    }

    printf("\nTotal Cost: %d\n\n", minCost);
}

int main(void)
{
    int matrix1[N][N] = {
        {3, 10, 7},
        {8, 5, 12},
        {4, 6, 9}
    };

    int matrix2[N][N] = {
        {15, 9, 4},
        {8, 7, 18},
        {6, 12, 11}
    };

    printf("Test Case 1:\n");
    solve(matrix1);

    printf("Test Case 2:\n");
    solve(matrix2);
```

Console | I/O | AI Agent | 175 ms

```
Test Case 1:
Optimal Assignment: (Worker 1, Task 3), (Worker 2, Task 2), (Worker 3, Task
1)
Total Cost: 16

Test Case 2:
Optimal Assignment: (Worker 1, Task 3), (Worker 2, Task 2), (Worker 3, Task
1)
Total Cost: 17
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:22 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c

#include <stdio.h>

#define MAX 4

int weights[MAX];
int values[MAX];
int n, capacity;
int bestValue;
int bestMask;

int totalValue(int mask)
{
    int i, sum = 0;

    for (i = 0; i < n; i++)
    {
        if (mask & (1 << i))
        {
            sum += values[i];
        }
    }

    return sum;
}

int totalWeight(int mask)
{
    int i, sum = 0;

    for (i = 0; i < n; i++)
    {
```

Console | I/O | AI Agent | 179 ms

```
Test Case 1:
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2:
Optimal Selection: [0, 1, 2]
Total Value: 12
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:22 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```
28  {
32      {
33          if (mask & (1 << i))
34          {
35              sum += weights[i];
36          }
37      }
38
39      return sum;
40  }
41
42  void knapsack(void)
43  {
44      int mask;
45      int value, weight;
46
47      bestValue = 0;
48      bestMask = 0;
49
50      for (mask = 0; mask < (1 << n); mask++)
51      {
52          weight = totalWeight(mask);
53          value = totalValue(mask);
54
55          if (weight <= capacity && value > bestValue)
56          {
57              bestValue = value;
58              bestMask = mask;
59          }
60      }
61  }
62
63  void solve(int itemWeights[], int itemValues[],
```

Console | I/O | AI Agent | 179 ms

```
Test Case 1:
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2:
Optimal Selection: [0, 1, 2]
Total Value: 12
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:22 09-10-2026

TOPIC 2.docx - Google Drive | C Online Compiler | Ask Gemini

onecompiler.com/c

OneCompiler | Upgrade | AI | C | Run | Save | Login

Main.c

```c
void solve(int itemWeights[], int itemValues[],
{
    int i;

    n = itemCount;
    capacity = cap;

    for (i = 0; i < n; i++)
    {
        weights[i] = itemWeights[i];
        values[i] = itemValues[i];
    }

    knapsack();

    printf("Optimal Selection: [");

    {
        int first = 1;

        for (i = 0; i < n; i++)
        {
            if (bestMask & (1 << i))
            {
                if (!first)
                    printf(", ");

                printf("%d", i);
                first = 0;
            }
        }
    }

```

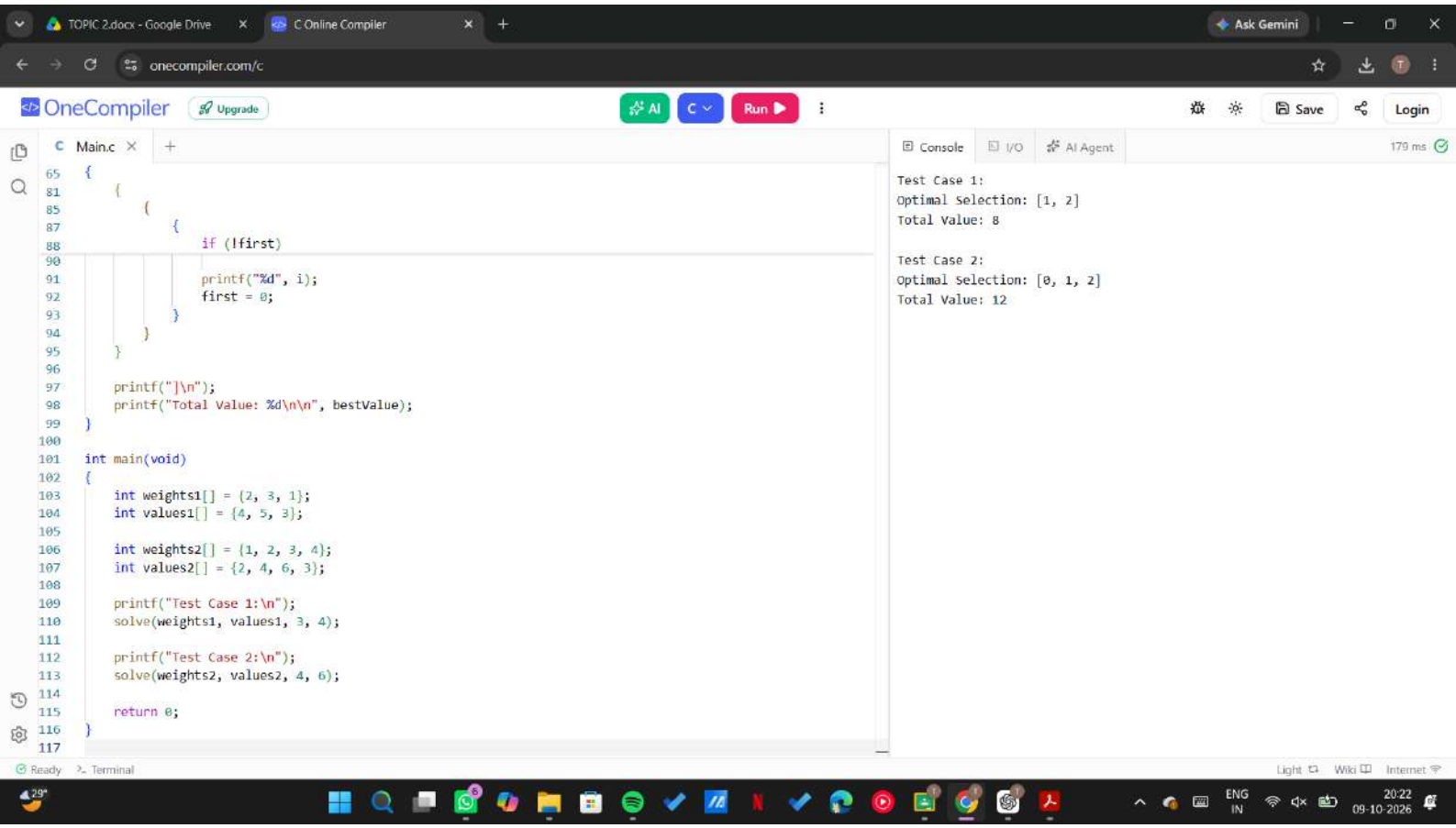
Console | I/O | AI Agent | 179 ms

```
Test Case 1:
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2:
Optimal Selection: [0, 1, 2]
Total Value: 12
```

Ready | Terminal | Light | Wiki | Internet

29° | ENG IN | 20:22 09-10-2026


TOPIC 2.docx - Google Drive
C Online Compiler
Ask Gemini
onecompiler.com/c
OneCompiler
Upgrade
AI
C
Run
Save
Login
Main.c
Console
I/O
AI Agent
179 ms
{
{
{
if (!first)
printf("%d", i);
first = 0;
}
}
}
printf("]\n");
printf("Total Value: %d\n\n", bestValue);
}
int main(void)
{
int weights1[] = {2, 3, 1};
int values1[] = {4, 5, 3};
int weights2[] = {1, 2, 3, 4};
int values2[] = {2, 4, 6, 3};
printf("Test Case 1:\n");
solve(weights1, values1, 3, 4);
printf("Test Case 2:\n");
solve(weights2, values2, 4, 6);
return 0;
}
Test Case 1:
Optimal Selection: [1, 2]
Total Value: 8
Test Case 2:
Optimal Selection: [0, 1, 2]
Total Value: 12
Ready
Terminal
Light
Wiki
Internet
29°
ENG
IN
20:22
09-10-2026